UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Sincronización de Tareas y Gestión de Recursos (Semáforos)

Práctica 6

Presenta:
Rojo Peña Manuel Ianluck

Profesor:
Javier León Cotonieto

Ayudante laboratorio:
Adrián Martínez Manzo

Grupo: 7004, 2026-2

Fecha de entrega:
13 abril, 2026

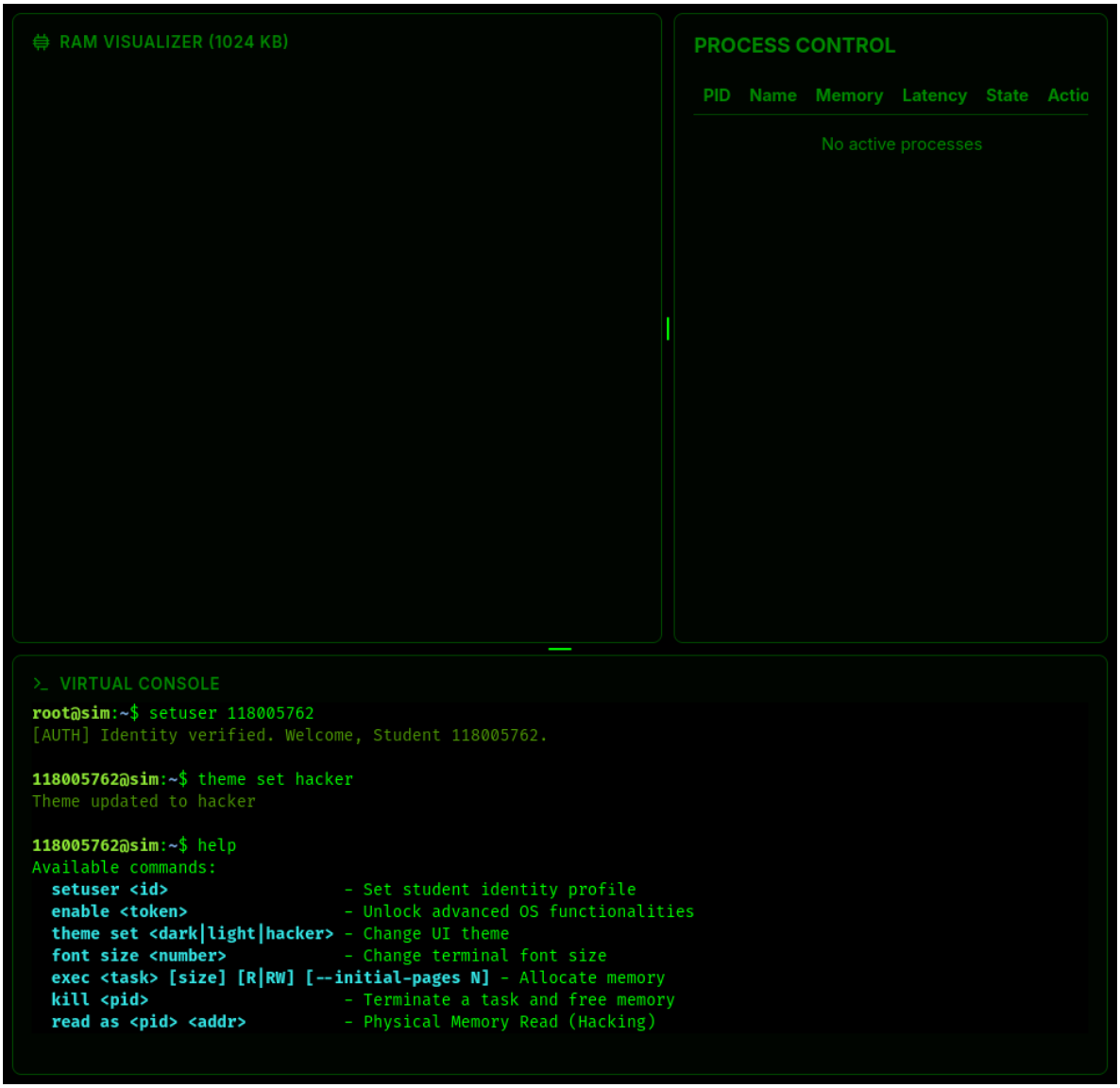
Introducción

La gestión de memoria es uno de los pilares fundamentales de los sistemas operativos modernos, y su evolución histórica refleja una respuesta progresiva a problemas concretos de seguridad, eficiencia y aprovechamiento de recursos. En esta práctica tendremos como propósito recorrer esa evolución desde dos perspectivas complementarias: la implementación directa en lenguaje C de los mecanismos fundamentales, y la experimentación interactiva mediante el simulador RAM-Sim.

A lo largo de las siguientes secciones se abordaron cinco etapas de esta evolución, partiendo del acceso directo a RAM, el esquema más primitivo donde los procesos operan sin protección ni abstracción sobre las direcciones físicas. Posteriormente se incorpora el mecanismo *Base-Límite*, que introduce la primera forma de aislamiento entre procesos mediante registros de hardware, el siguiente paso es la paginación básica, que elimina la fragmentación externa al dividir la memoria en marcos de tamaño fijo y habilita la compartición segura entre procesos. Después se analiza el papel del hardware especializado a través de la **MMU** y el **TLB**, cuya función es mitigar el costo de la traducción de direcciones. Finalmente se estudia la paginación bajo demanda, que permite ejecutar procesos cuyo tamaño excede la RAM disponible mediante el uso del disco como almacenamiento secundario.

Cada parte combina la programación de funciones en C con la validación y observación del comportamiento en el simulador, lo que permite contrastar la teoría con los efectos concretos sobre métricas como la latencia de ejecución, el número de ciclos, los fallos de página y la tasa de aciertos del **TLB**. El objetivo final es comprender por qué los sistemas operativos contemporáneos dependen de una estrecha colaboración entre software y hardware para ofrecer a cada proceso la ilusión de un espacio de memoria propio, protegido, continuo y potencialmente mayor que la memoria física real.

Parte 0. Configuración del simulador



Parte 1. RAM en modo directo

1.1 Carga Inicial:

Verificamos que, efectivamente utilizamos los **1024 kb**:

$$500 + 175 + 249 + 100 = 1024 \text{ KB}$$

Al ejecutar secuencialmente los comandos `exec taskA 500`, `exec taskB 175`, `exec taskC 249` y `exec taskD 100`, se observan en el visualizador de memoria los cuatro distintos colores de bloques asignados de forma contigua, sin dejar espacio libre alguno que corresponden a estas tareas.

1.2. Gestión de Procesos:

En la imagen se aprecia como se terminan los procesos `taskB` (`PID=101`) y `taskC` (`PID=102`) mediante el comando `kill`, lo que libera un total de `175 + 249 = 424 KB` de memoria. Después se intenta cargar una nueva tarea, `taskE`, con un tamaño de **400 KB**, el cual es menor que los **424 KB** liberados.

El resultado es que `taskE` sí se carga exitosamente, el simulador asigna `taskE` en el espacio contiguo que antes ocupaban `taskB` y `taskC` quedando un pequeño hueco de **24 KB** entre `taskE` y `taskD`.
Esto se debe a que, en modo directo, el sistema busca el primer espacio contiguo de tamaño suficiente, como los bloques liberados por `taskB` y `taskC` eran continuos entre sí, formaron un único bloque de **424 KB** donde `taskE` cabe sin problema, lo que posibilita la asignación de memoria.

1.3. Fragmentación Externa:

Se terminan los procesos `taskE` y `taskD` para liberar la memoria, luego se recargan `taskB` (**175 KB**), `taskC` (**249 KB**) y `taskD` (**100 KB**), restaurando el estado original. Acto seguido, se eliminan únicamente `taskB` y `taskD`, dejando un hueco de **175 KB** y otro de **100 KB**, siendo en total **275 KB** libres de memoria.

Al intentar cargar `taskF` con **200 KB**, un tamaño que es mayor a cualquiera de los huecos pero menor que la suma de ambos, el resultado es que el simulador muestra el error:

```
[KERNEL ERROR] Out of Memory: No contiguous space for 200 KB.
```

A pesar de disponer de **275 KB** libres en total, no existe un bloque único lo suficientemente grande para alojar la tarea solicitada.

Este fenómeno se denomina *fragmentación externa*: se produce cuando la memoria libre total es suficiente para satisfacer una solicitud de asignación, pero dicha memoria se encuentra dispersa en múltiples bloques no contiguos (justo como lo dice el mensaje de error), ninguno de los cuales tiene el tamaño requerido por sí solo.
Para un sistema operativo, la fragmentación externa representa un problema serio de eficiencia y disponibilidad. Aunque existe memoria física sin utilizar, el sistema es incapaz de aprovecharla, lo que conduce a rechazos de solicitudes legítimas y a una subutilización de los recursos de hardware. En escenarios de larga ejecución, donde los procesos se crean y destruyen continuamente, la fragmentación tiende a agravarse progresivamente.
Las soluciones clásicas a este problema incluyen la compactación de memoria (reubicar los procesos activos para consolidar los huecos libres en un solo bloque contiguo), aunque

esta operación es costosa en términos de tiempo. La solución más adoptada por los sistemas operativos modernos es la paginación, que divide la memoria en marcos de tamaño fijo y elimina el requisito de contigüidad, tema que se abordará en las secciones posteriores de esta práctica.

1.4. Protección y Seguridad:

Se eliminan todos los procesos y se inician dos nuevos procesos, `taskA` y `taskB`, de **10 KB** cada uno, en el simulador, los primeros 10 bloques de memoria (de color amarillo) quedan asignados a `taskA` y los siguientes 10 (de color azul claro) a `taskB`.

Se realizaron las siguientes operaciones:

- `read as 110 0`: el proceso B lee exitosamente la posición de memoria 0 de A, lo que indica el problema de seguridad.
- `write as 110 0 1`: el proceso B escribe el valor 1 en la posición 0, la operación se realiza sin restricciones, otra indicación de un problema grave de seguridad.

Los resultados obtenidos revelan una falla de seguridad crítica en el modo de acceso directo a memoria, se puede observar en el simulador que aparece un bloque de color azul (correspondiente a `taskB`) en la sección de memoria que debería ser exclusiva de `taskA`, y un bloque amarillo (correspondiente a `taskA`) en la zona de `taskB`. Esto indica que no existe ningún mecanismo de aislamiento entre los espacios de direcciones de los procesos.

En un sistema sin protección de memoria, cualquier proceso puede leer o escribir en cualquier posición de la memoria física, incluyendo las regiones asignadas a otros procesos o incluso al propio sistema operativo. Lo que implica vulnerabilidades graves pues un proceso malicioso (o simplemente defectuoso) podría corromper los datos de otro proceso, inyectar código ejecutable en su espacio de memoria, leer información confidencial como contraseñas o claves de cifrado, o incluso modificar las estructuras internas del kernel, comprometiendo la estabilidad y seguridad de todo el sistema.

Con el simulador nos muestra de manera contundente la necesidad de implementar mecanismos de protección de memoria, como los registros *Base* y *Límite* que se explorarán en la **Parte 2** de esta práctica, o esquemas más avanzados basados en paginación con bits de protección por página.

1.5 Bibliotecas Compartidas:

El último escenario planteado propone imaginar que los procesos A y B comparten una biblioteca de código alojada en la primera posición de memoria de cada uno. La pregunta es si el sistema actual permitiría compartir ese bloque para ahorrar espacio, y qué riesgos implicaría.

En el modo de acceso directo, cada proceso recibe un bloque contiguo de memoria física propio, no existe un mecanismo de indirección (como una tabla de páginas) que permita que dos procesos mapeen direcciones lógicas distintas a una misma región física. Por lo tanto, el sistema actual no tiene la capacidad de compartir un bloque de memoria de manera estructurada entre dos procesos.

Si se intentara lograr la compartición manualmente por ejemplo, haciendo que ambos procesos lean la misma dirección física, se enfrentarían múltiples riesgos:

- Sin distinción entre código y datos: al no existir bits de protección que marquen una región como de solo lectura, cualquiera de los dos procesos podría sobrescribir el código compartido, corrompiendo la biblioteca para ambos.
- Sin control de acceso: no hay forma de restringir qué procesos pueden acceder a la región compartida. Cualquier proceso en el sistema (no solo A y B) podría leer o modificar ese bloque.

- Fragmentación agravada: al obligar a los procesos a ubicarse de forma que compartan una dirección física específica, las restricciones de posicionamiento empeorarían aún más el problema de fragmentación externa.
- Falta de transparencia: los procesos necesitarían conocer las direcciones físicas exactas de la biblioteca compartida, violando el principio de abstracción y haciendo que el código sea completamente dependiente de la disposición física de la memoria.

La compartición segura de bibliotecas requiere un sistema de memoria virtual con paginación, donde múltiples procesos pueden mapear páginas virtuales distintas al mismo marco físico, con permisos de solo lectura/ejecución. Este mecanismo, que se explora en la **Parte 3**, permite a los sistemas operativos modernos compartir bibliotecas como `libc` o `libm` entre decenas de procesos sin duplicar el código en memoria y sin comprometer la seguridad.

Parte 2.

2.1. Implementación:

Se programó la función `base_limit` en el archivo `vm_bl.c`, la lógica implementada realiza los siguientes pasos: primero, recorre el arreglo de procesos registrados buscando aquel cuyo `PID` coincida con el solicitado. Si lo encuentra, verifica que la dirección física solicitada se encuentre dentro del rango válido del proceso, es decir, que sea mayor o igual a la base y estrictamente menor que `base + limite`. La función retorna 1 si el acceso es válido, 0 si constituye una violación de límites, o -1 si el proceso no fue encontrado. Adicionalmente, se completaron las funciones `ram_write` y `ram_read` para que, en caso de acceso válido, operen directamente sobre el arreglo global de RAM en lugar de usar valores placeholder.

```c
#include <stdio.h>
#include "vm_bl.h"

#define MAX_PROCESSES 10

process_t processes[MAX_PROCESSES];
int process_count = 0;

/* Registramos un nuevo proceso con su rango de memoria */
void
assign_process(int pid,
           int base,
           int limit)
{
  if (process_count < MAX_PROCESSES) {
    processes[process_count].pid = pid;
    processes[process_count].base = base;
    processes[process_count].limit = limit;
    process_count++;
  } else
    fprintf(stderr, "Error: Maximum number of processes reached\n");
}

/* Verificamos si un acceso a memoria es valido para un proceso */
int
base_limit(int pid,
       int address)
{
  //Buscar el proceso por PID
  for (int i = 0; i < process_count; i++) {
    if (processes[i].pid == pid) {
      //Verificar si la direccion esta dentro del rango [base, base + limite)
      if (address >= processes[i].base && address < processes[i].base + processes[i].limit)
        return 1; //Acceso valido
```

```c
      else
        return 0; //Violacion de limite
    }
  }
  return -1; //El proceso no existe
}

/*
 * Escribe un byte en RAM previa verificacion Base-Limit,
 * solo se ejecuta la escritura si base_limit valida el acceso
 */
void
ram_write(int pid,
      int address,
      unsigned char value)
{
  int result = base_limit(pid, address);
  if (result == 1) {
    ram[address] = value;
    printf("WRITE: Process %d wrote %02X to 0x%03X\n", pid, value, address);
  } else if (result == 0)
    printf("ERROR: Process %d access violation at 0x%03X (Base-Limit)\n", pid, address);
  else
    printf("ERROR: Process %d not found\n", pid);
}

/* Lee un byte de RAM previa verificacion Base-Limite */
unsigned char
ram_read(int pid,
     int address)
{
  int result = base_limit(pid, address);
  if (result == 1) {
    unsigned char value = ram[address];
    printf("READ: Process %d read %02X from 0x%03X\n", pid, value, address);
    return value;
  } else if (result == 0)
    printf("ERROR: Process %d access violation at 0x%03X (Base-Limit)\n", pid, address);
  else
    printf("ERROR: Process %d not found\n", pid);
  return 0;
}
```

Código 1: Implementación base limit `vm_bl.c`

2.2 Validación:

Al compilar y ejecutar el validador en el entorno Docker proporcionado, con el cambio `#define STUDENT_ID` con `118005762` mi número de cuenta correspondiente, todas las pruebas fueron aprobadas satisfactoriamente.

```
 [src/] > docker run validadorr
============================================
   RAM SIMULATION AUTOMATED VALIDATOR
============================================
VALIDATING FOR STUDENT ID: 118005762

PHASE: Base-Limit Protection (mv-bl)
    [OK]   Check #1 passed.
    [OK]   Check #2 passed.
    RESULT: PASS
        TOKEN: ECBBFA9E2D279932E26363A308BC198B

PHASE: Basic Paging (pag-basic)
    [FAIL] Check #1: Incorrect page allocation for process size 7 with p
age size 4
    [FAIL] Check #2: Failed to handle memory exhaustion correctly
    RESULT: FAIL (Please review the errors above)

PHASE: MMU Basic Translation (mmu-hw)
    [FAIL] Check #1: Incorrect virtual-to-physical translation calculati
on
    [FAIL] Check #2: Failed to report error for unmapped page
    RESULT: FAIL (Please review the errors above)

PHASE: TLB with LRU/FIFO (tlb-fast)
    [FAIL] Check #1: Failed to insert page into TLB on first access
    [FAIL] Check #2: Failed to identify TLB HIT on subsequent access
    [FAIL] Check #3: Incorrect TLB eviction logic (LRU failed)
    RESULT: FAIL (Please review the errors above)

PHASE: Demand Paging (pg-od)
    [FAIL] Check #1: Failed to pin Page Table in RAM
```

2.3 Habilitación:

La validación del sistema generó un código el cuál se introdujo en el simulador mediante el comando `enable [code]`, lo que desbloqueó el módulo *Base-Límite* en el entorno de

simulación para tener acceso a la verificación de los registros Base y Limite antes de ser ejecutados.

```
root@sim:~$ setuser 118005762
[AUTH] Identity verified. Welcome, Student 118005762.

118005762@sim:~$ theme set hacker
Theme updated to hacker

118005762@sim:~$ enable ECBBFA9E2D279932E26363A308BC198B
[SYSTEM ADMIN] Access GRANTED. Module [mv-bl] unlocked for 118005762.

118005762@sim:~$
```

2.4 Experimentación:

Al repetir los ejercicios de la **Parte 1** con el nuevo módulo activo, el comportamiento es el mismo que, el cambio fundamental se observa al repetir el ejercicio de protección (1.4) pues, a diferencia del modo directo, donde un proceso podía leer y escribir libremente en la memoria de otro sin restricción alguna, con *Base-Límite* activo el sistema ahora intercepta y bloquea los accesos no autorizados. Al intentar que el proceso A acceda o modifique la memoria del proceso B y viceversa, el simulador genera el siguiente error:

```
[HW FAULT] Access Denied: Segmentation Violation. Physical Addr 11 is out of bounds.
```

Lo que indica que se detectó que la dirección física 11 (que pertenece al espacio de `taskB`) se encuentra fuera del rango permitido para el proceso que intentó el acceso

(`taskA`). La operación es abortada inmediatamente, protegiendo la integridad de los datos del otro proceso.

2.5 Análisis de Ejecución:

```
>_ VIRTUAL CONSOLE

118005762@sim:~$ switch 101
[SYS] Context switch successful. Active CPU context: PID 101.

118005762@sim:~$ step
[CPU] Executing IP 1/16: READ ADDR_OFFSET 4 KB
  [CPU] IP advanced to 1

118005762@sim:~$ step
[CPU] Executing IP 2/16: READ ADDR_OFFSET 6 KB
  [CPU] IP advanced to 2

118005762@sim:~$ step
[CPU] Executing IP 3/16: READ ADDR_OFFSET 8 KB
  [CPU] IP advanced to 3

118005762@sim:~$ step
[CPU] Executing IP 4/16: READ ADDR_OFFSET 6 KB
  [CPU] IP advanced to 4

118005762@sim:~$ step
[CPU] Executing IP 5/16: READ ADDR_OFFSET 9 KB
  [CPU] IP advanced to 5

118005762@sim:~$ step
[CPU] Executing IP 6/16: READ ADDR_OFFSET 4 KB
  [CPU] IP advanced to 3
  [CPU] Branch Loop taken. IP backed-up to 3.

118005762@sim:~$ step
[CPU] Executing IP 4/16: READ ADDR_OFFSET 6 KB
  [CPU] IP advanced to 4

118005762@sim:~$ step
[CPU] Executing IP 5/16: READ ADDR_OFFSET 9 KB
  [CPU] IP advanced to 2
  [CPU] Branch Loop taken. IP backed-up to 2.

118005762@sim:~$
```

Se seleccionó el proceso con `PID=101` desde el panel de control y se ejecutó instrucción por instrucción mediante el botón `step`, el proceso contiene un total de 16 instrucciones

programadas; sin embargo, debido a la presencia de instrucciones de salto condicional (*Branch Loop*), el número real de pasos ejecutados fue considerablemente mayor.

A lo largo de la ejecución completa, el *Instruction Pointer* (**IP**) avanzó de forma no lineal, en múltiples ocasiones, al encontrar una instrucción de salto cuya condición se cumplía, el IP retrocedió a una instrucción anterior, repitiendo secciones del código. Este comportamiento de bucle se observó en varios puntos del programa, por ejemplo, al ejecutar la instrucción en la posición 6, el IP retrocedió a la posición 3, al ejecutar la posición 5, retrocedió a la 2, y al ejecutar la posición 9, retrocedió a la 5, entre otros casos.

En total, la ejecución completa del proceso requirió aproximadamente 38 pasos (`steps`) para completar las 16 instrucciones del programa, esto significa que, en promedio, cada instrucción original se ejecutó alrededor de 2.4 veces debido a los bucles.
Cada instrucción ejecutada bajo el esquema Base-Límite implica un costo adicional respecto al modo directo, antes de que la CPU pueda realizar el acceso real a la memoria (lectura o escritura), el hardware debe llevar a cabo una verificación de protección: comparar la dirección física solicitada contra los registros Base y Límite del proceso activo. Esta comparación, aunque rápida a nivel de hardware (se realiza mediante circuitos comparadores), introduce una latencia adicional en cada acceso a memoria.

En el modo directo de la **Parte 1**, un acceso a memoria se traduce directamente a una operación sobre la RAM física sin ningún paso intermedio. Con *Base-Límite*, el flujo se extiende de la siguiente manera: primero, la CPU genera la dirección física, segundo, el hardware compara esta dirección con el registro Base para verificar que no sea inferior al inicio del segmento, tercero, compara con `Base + Límite` para verificar que no exceda el final del segmento, y solo si ambas comparaciones son satisfactorias, se permite el acceso efectivo a la RAM.

Esta latencia adicional es el precio que se paga por la seguridad, sin embargo, dado que las comparaciones se realizan en hardware dedicado (no en software), el *overhead* es mínimo, típicamente uno o dos ciclos de reloj adicionales por acceso; se trata de un costo ampliamente justificado por las garantías de aislamiento que proporciona, como se demostró en la sección de experimentación.

Parte 3.

3.1 Habilitación:

```
[src/] > docker run validadorr
==========================================
   RAM SIMULATION AUTOMATED VALIDATOR
==========================================
VALIDATING FOR STUDENT ID: 118005762

PHASE: Base-Limit Protection (mv-bl)
    [OK]   Check #1 passed.
    [OK]   Check #2 passed.
    RESULT: PASS
        TOKEN: ECBBFA9E2D279932E26363A308BC198B

PHASE: Basic Paging (pag-basic)
    [OK]   Check #1 passed.
    [OK]   Check #2 passed.
    RESULT: PASS
        TOKEN: 65F4FB5B4B5B6718DACF6D3575819452

PHASE: MMU Basic Translation (mmu-hw)
    [FAIL] Check #1: Incorrect virtual-to-physical translation calculati
on
    [FAIL] Check #2: Failed to report error for unmapped page
    RESULT: FAIL (Please review the errors above)

PHASE: TLB with LRU/FIFO (tlb-fast)
    [FAIL] Check #1: Failed to insert page into TLB on first access
    [FAIL] Check #2: Failed to identify TLB HIT on subsequent access
    [FAIL] Check #3: Incorrect TLB eviction logic (LRU failed)
    RESULT: FAIL (Please review the errors above)

PHASE: Demand Paging (pg-od)
    [FAIL] Check #1: Failed to pin Page Table in RAM
```

Se programó la función `basic_paging` en el código de C proporcionado, se validó mediante el validador en el entorno Docker, y se obtuvo el *Enabler Code* correspondiente. Se ingresó el código en el simulador con el comando `paging enable`, lo que activó el módulo de paginación básica, se verificó previamente que no existieran procesos activos en el sistema, como lo requiere el procedimiento.

```c
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <string.h>
#include "paging_basic.h"

/* Simulador de paginacion basica.
 * Leemos comandos desde stdin y gestionamos la asignacion de marcos (frames) a procesos
 */
void
simulate_basic_paging()
{
  int page_size = 0;    //Tamano de cada pagina/marco en bytes
  int ram_size = 0;     //Tamano total de la RAM en bytes
  int total_frames = 0; //Numero total de marcos en el sistema
  int free_frames = 0;  //Marcos actualmente disponibles
  char line[256];

  while (fgets(line, sizeof(line), stdin)) {
    line[strcspn(line, "\r\n")] = 0;
    if (line[0] == '\0' || line[0] == '#')
      continue;

    if (strncmp(line, "PAGE_SIZE", 9) == 0) {
      sscanf(line, "PAGE_SIZE %d", &page_size);
      printf("SYSTEM: Page Size set to %d bytes\n", page_size);
    } else if (strncmp(line, "RAM_SIZE", 8) == 0) {
      sscanf(line, "RAM_SIZE %d", &ram_size);
      //Solo se cuentan marcos completos
      total_frames = ram_size / page_size;
      free_frames = total_frames;
      printf("SYSTEM: RAM Size set to %d bytes (%d frames)\n", ram_size, total_frames);
    } else if (strncmp(line, "PROCESS PID", 11) == 0) {
      int pid;
      int size;
      if (sscanf(line, "PROCESS PID %d SIZE %d", &pid, &size) == 2) {
    int pages_needed = (size + page_size - 1) / page_size;
    if (pages_needed <= free_frames) {
      // Actualizar frames libres
      free_frames -= pages_needed;
      printf("PROCESS: PID %d (Size: %d) -> %d pages allocated. Frames left: %d\n",
          pid, size, pages_needed, free_frames);
    } else {
      // Notificar falla por falta de memoria
      printf("PROCESS: PID %d (Size: %d) -> %d pages needed. FAILED (Not enough memory, frames left: %d)\n",
          pid, size, pages_needed, free_frames);
    }
      }
    }
  }
}
```

Código 2: Implementación paging basic `paging_basic.c`

3.2 Pruebas de Carga:

Al repetir los escenarios de carga de la **Parte 1** con paginación activa, se observa un cambio fundamental en la gestión del espacio libre. En la **Parte 1**, al eliminar un proceso intermedio y cargar uno de mayor tamaño que el hueco dejado, el sistema rechazaba la operación por falta de espacio contiguo (fragmentación externa).

Ahora, con paginación, este escenario se resuelve de forma transparente, como se puede apreciar en la evidencia visual, al cargar un nuevo proceso cuyo tamaño excede el espacio dejado por el proceso B eliminado, el sistema asigna tantas páginas como caben en el hueco intermedio y coloca las páginas restantes al final de la memoria, después del último marco ocupado. El proceso queda distribuido en marcos no contiguos, pero esto resulta completamente transparente para el programa en ejecución: desde su perspectiva, su espacio de direcciones virtuales sigue siendo lineal y continuo.

Este resultado confirma que la paginación elimina por completo la fragmentación externa. La única forma de fragmentación que persiste es la fragmentación interna: si el tamaño del proceso no es múltiplo exacto del tamaño de página, la última página asignada queda parcialmente desaprovechada. Sin embargo, este desperdicio está acotado a como máximo un marco por proceso, lo que representa una mejora sustancial frente al problema ilimitado de la fragmentación externa.

Respecto a la compartición de código, la paginación abre la posibilidad de que múltiples procesos mapeen páginas virtuales distintas al mismo marco físico, lo que permite, por

ejemplo, que dos procesos compartan una biblioteca de código cargada en un solo marco, sin duplicar su contenido en RAM.

3.3 Tamaño de Página:

El simulador permite configurar tres tamaños de página distintos para una memoria física total de **1024 KB**: **4 KB** (`256 marcos`), **8 KB** (`128 marcos`) y **16 KB** (`64 marcos`). El tamaño de página influye directamente en varios aspectos del rendimiento y la eficiencia del sistema:

Con páginas más grandes, la probabilidad de desperdiciar espacio en la última página de cada proceso aumenta, por ejemplo, un proceso de **17 KB** con páginas de **16 KB** necesita 2 páginas (por **32 KB** asignados), desperdiciando **15 KB**. El mismo proceso con páginas de **4 KB** necesita 5 páginas (**20 KB**), desperdiciando solo **3 KB**. En promedio, la fragmentación interna esperada es de la mitad del tamaño de página por proceso, **2 KB** para páginas de **4 KB**, **4 KB** para páginas de **8 KB**, y **8 KB** para páginas de **16 KB**.

Páginas más pequeñas generan tablas de páginas más grandes, ya que cada proceso necesita más entradas para cubrir su espacio de direcciones. Con `256 marcos` (páginas de **4 KB**), la tabla de páginas de un proceso grande puede contener cientos de entradas, lo que consume memoria adicional y aumenta el tiempo de búsqueda durante la traducción de direcciones. En cambio, con `64 marcos` (páginas de **16 KB**), las tablas son mucho más compactas.
Las operaciones de carga y descarga de páginas entre RAM y disco (relevantes para la paginación bajo demanda de la **Parte 5**) son más eficientes con páginas grandes, ya que el costo fijo de cada transferencia de disco se amortiza sobre un bloque de datos mayor. Con páginas pequeñas, se requieren más transferencias individuales para mover la misma cantidad de datos.

Lo que nos lleva a que, las páginas pequeñas (**4 KB**) son preferibles cuando los procesos son de tamaño variado y se desea minimizar el desperdicio de memoria, o cuando la localidad de referencia es fina (los accesos se concentran en pequeñas regiones de memoria). Las páginas grandes (**16 KB**) son más adecuadas para cargas de trabajo con accesos secuenciales a grandes volúmenes de datos, donde la reducción en el tamaño de la tabla de páginas y el menor número de fallos de página compensan el mayor desperdicio por fragmentación interna, el tamaño de 8 KB ofrece un equilibrio intermedio.

3.4 Compartición Segura:

Se configuraron dos procesos en el simulador con paginación activa y tamaño de página de **8 KB**. Se ejecutaron los siguientes comandos:

Se asignó `taskA` con `PID=106` y un segmento de **100 KB** (13 páginas de **8 KB**).

```
exec taskA 100
```

Se asignó `taskB` con `PID=107` y un segmento de **69 KB** (9 páginas de **8 KB**).

```
exec taskB 69
```

El proceso A pudo leer su propia memoria sin problema, mientras que el proceso B recibió un error al intentar ac ceder a un marco que no le pertenece:

```
read as 106 1
```

Resultado:

```
[HW READ] Data at Physical Block 1: [Belongs to PID 106]. Lectura exitosa.
```

```
read as 107 1
```

Resultado:

```
[HW FAULT] Access Denied: Not owner or co-owner of Frame 1. Acceso denegado correctame
```

A continuación, se estableció la compartición de la página virtual 1 (`VPN 1`) del proceso A con el proceso B. Primero se configuraron los permisos y luego se agregó al copropietario:

```
chmod 1 RW-
[SYS] Permissions for VPN 1 updated to RW-

add-owner 1 107
[KERNEL] Shared VPN 1 (PFN 8) with Process 107.
```

El kernel mapea la página virtual 1 al marco físico 8 y registra al proceso 107 como copropietario.

Una vez establecida la compartición, se verificó que el proceso B puede ahora acceder al marco compartido:

```
read as 107 8
```

Resultado:

```
[HW READ] Data at Physical Block 8: [Belongs to PID 106].
```

La lectura es exitosa. Nótese que el dato sigue indicando que pertenece al `PID=106` (el propietario original), pero el proceso `PID=107` tiene acceso como copropietario.

Finalmente, se revocó el acceso compartido y se verificó que la revocación surte efecto inmediato:

```
remove-owner 1 107
[KERNEL] Removed sharing of VPN 1 from Process 107.


read as 107 8
```

Resultado:

```
[HW FAULT] Access Denied: Not owner or co-owner of Frame 8.
```

El acceso fue denegado tras la revocación.

Este experimento demuestra que la paginación permite compartir marcos físicos de memoria entre procesos de manera controlada y segura. A diferencia del modo directo (**Parte 1**), donde la compartición era imposible de gestionar sin riesgos, el sistema de paginación ofrece tres niveles de control: la gestión de propietarios (`add-owner`/`remove-owner`) determina qué procesos pueden acceder a cada marco, los permisos por página (`chmod`) controlan el tipo de acceso permitido (lectura, escritura, ejecución), y la revocación inmediata garantiza que los cambios de política se aplican sin retardo.

3.5 Latencia:

Se seleccionó el proceso con `PID=106` y se ejecutó instrucción por instrucción mediante `step`, de forma análoga al análisis de la **Parte 2**. El proceso contiene 16 instrucciones, y su ejecución completa requirió aproximadamente 50 pasos debido a las instrucciones de salto condicional (*Branch Loop*).

La diferencia más notable respecto al modo *Base-Límite* de la Parte 2 es el incremento significativo en la latencia de cada instrucción. En cada paso ejecutado, el simulador muestra dos mensajes característicos que no aparecían en las partes anteriores (aunque, se muestra en el simulador que la latencia es de `0 ms/instr`):

- `[OS] Software Page Table Walk in progress ..... [DONE]`: indica que el sistema operativo está recorriendo la tabla de páginas del proceso para traducir la dirección virtual (**VPN**) a la dirección física (**PFN**) correspondiente. Aqui es donde muy brevemente se ve como tarda en ejecutar este paso.
- `[MMU MISS] TLB Miss, walking Page Table...`: confirma que no existe una entrada en caché (**TLB**) para esa traducción, por lo que es necesario realizar el recorrido completo de la tabla de páginas.

Es importante destacar que el 100% de los accesos resultaron en `TLB Miss`, ya que el módulo de paginación básica opera sin **TLB**, al no existir una caché de traducciones recientes, cada acceso a memoria requiere un recorrido completo de la tabla de páginas en memoria, lo que constituye uno o más accesos adicionales a RAM por cada instrucción.

En el modo directo, cada acceso a memoria se traduce directamente a una operación sobre la RAM física sin ninguna verificación ni traducción, lo que constituye el caso

más rápido. En el modo *Base-Límite*, se añaden dos comparaciones de hardware por acceso (verificar que la dirección esté dentro del rango del proceso), lo que introduce un *overhead* mínimo de uno o dos ciclos. En la paginación básica, cada acceso requiere un *Page Table Walk* completo, el sistema debe acceder a la tabla de páginas en memoria, buscar la entrada correspondiente al **VPN** solicitado, extraer el **PFN**, y recién entonces realizar el acceso efectivo a la RAM, esto duplica o triplica el número efectivo de accesos a memoria por cada instrucción del programa.
Una diferencia importante respecto a las partes anteriores es que, a partir de la activación de la paginación, el simulador habilita el panel de estadísticas *Page Tables Overhead*, que cuantifica el costo acumulado de los recorridos de tabla de páginas durante la ejecución. En los modos directo y *Base-Límite* este panel no existía porque no había tablas de páginas que consultar, todo acceso a memoria era inmediato o requería solo una comparación de hardware. Durante la ejecución con paginación básica, el indicador muestra un valor del 4.7% con 12 de 256 páginas registradas como *overhead*, reflejando el costo estructural de mantener y recorrer las tablas de páginas en cada traducción. Este valor crece proporcionalmente al número de procesos activos y a la frecuencia de accesos a memoria, y motiva directamente la introducción de la **MMU** y el **TLB** en la **Parte 4** como mecanismos para mitigar ese *overhead*.

Parte 4.

```
 [src/] > docker run validadorr
==========================================
   RAM SIMULATION AUTOMATED VALIDATOR
==========================================
VALIDATING FOR STUDENT ID: 118005762

PHASE: Base-Limit Protection (mv-bl)
    [OK]   Check #1 passed.
    [OK]   Check #2 passed.
    RESULT: PASS
        TOKEN: ECBBFA9E2D279932E26363A308BC198B

PHASE: Basic Paging (pag-basic)
    [OK]   Check #1 passed.
    [OK]   Check #2 passed.
    RESULT: PASS
        TOKEN: 65F4FB5B4B5B6718DACF6D3575819452

PHASE: MMU Basic Translation (mmu-hw)
    [OK]   Check #1 passed.
    [OK]   Check #2 passed.
    RESULT: PASS
        TOKEN: E0EDC84E298C9A0B37976336D6C4C176

PHASE: TLB with LRU/FIFO (tlb-fast)
    [OK]   Check #1 passed.
    [FAIL] Check #2: Failed to identify TLB HIT on subsequent access
    [FAIL] Check #3: Incorrect TLB eviction logic (LRU failed)
    RESULT: FAIL (Please review the errors above)

PHASE: Demand Paging (pg-od)
    [FAIL] Check #1: Failed to pin Page Table in RAM
```

Escenario A: TLB de una ranura

Se programó la función `mmu_translate` en el archivo `mmu_basic.c`, la lógica implementada calcula el número de página como el cociente entero de la dirección virtual entre el tamaño de página, y el *offset* como el residuo. Posteriormente busca el marco correspondiente en la tabla de páginas y calcula la dirección física como:

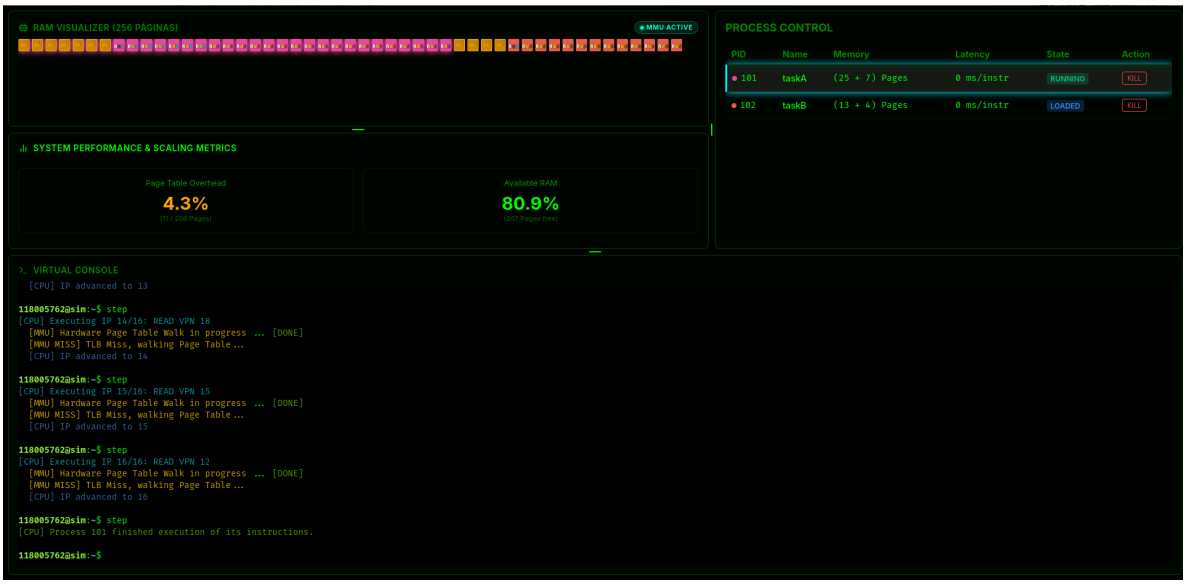
$$frame \times page_size + offset$$

Se validó el código y se habilitó el módulo en el simulador.

```c
#include <stdio.h>
#include "mmu.h"
#include "tlb.h"

#define PAGE_TABLE_SIZE 256
//Tabla de paginas con la que mapeamos VPN -> PFN
int page_table[PAGE_TABLE_SIZE];
int use_tlb = 0; //Flag to enable TLB behavior

//Inicializa la tabla de paginas (todas las entradas invalidas) y el TLB
void
init_mmu()
{
  for (int i = 0; i < PAGE_TABLE_SIZE; i++)
    page_table[i] = -1;
  init_tlb();
}

//Activa el uso del TLB como cache de traducciones
void
enable_tlb()
{
  use_tlb = 1;
}

/* Establece un mapeo pagina -> marco en la tabla */
void
set_page_table_entry(int page,
             int frame)
{
  if (page >= 0 && page < PAGE_TABLE_SIZE) {
    page_table[page] = frame;
    printf("MMU: Page %d mapped to Frame %d\n", page, frame);
  } else
    fprintf(stderr, "Error: Page index out of bounds\n");
}

/*
 * Traduce una direccion virtual a direccion fisica
 * Procedimiento:
 *   1. Calcula VPN = virtual_addr / page_size
 *   2. Calcula offset = virtual_addr % page_size
 *   3. Si el TLB esta activo, busca ahi primero (rapido, 1 ciclo)
 *   4. Si es TLB miss, consulta la tabla de paginas (Page Table Walk)
 *   5. Si se encontro en tabla y TLB activo, inserta la traduccion en TLB
 *   6. Direccion fisica = frame * page_size + offset
 */
int
```

```c
mmu_translate(int virtual_addr,
          int page_size)
{
  if (page_size <= 0)
    return -1;
  int page = virtual_addr / page_size;   //Numero de pagina virtual
  int offset = virtual_addr % page_size; //Desplazamiento dentro de la pagina
  int frame = -1;

  //Paso 1: Consultar TLB si esta habilitado
  if(use_tlb)
    frame = tlb_lookup(page);
  //Paso 2: Si TLB miss (o TLB deshabilitado), recorrer tabla de paginas
  if (frame == -1) {
    if (page >= 0 && page < PAGE_TABLE_SIZE)
      frame = page_table[page];
    if (frame != -1 && use_tlb)
      tlb_insert(page, frame);
  }

  if (frame != -1) {
    //Calculamos direccion fisica, base del marco + offset
    int physical_addr = frame * page_size + offset;
    printf("TRANSLATE: Virtual 0x%04X -> Physical 0x%04X (Frame %d)\n",
        virtual_addr, physical_addr, frame);
    return physical_addr;
  } else {
    printf("TRANSLATE ERROR: Page %d is not mapped\n", page);
    return -1;
  }
}
```

Código 3: Implementación mmu sin tlb `mmu.c`



Se ejecutó un proceso `taskA` de **100 KB** instrucción por instrucción mediante `step`. La ejecución completa requirió aproximadamente 26 pasos para las 16 instrucciones del programa. Esto representa una mejora significativa respecto a los casi 50 pasos de la paginación básica, lo que indica que la **MMU** por hardware reduce el número de iteraciones de los bucles al ejecutar la traducción más rápidamente.

Se observa que el 100% de los accesos resultaron en **TLB Miss**, al igual que en la **Parte 3**, esto es completamente esperable, ya que en este escenario el **TLB** aún no está habilitado. La **MMU** realiza un *Hardware Page Table Walk* en cada acceso, aunque este recorrido es más rápido que el *Software Page Table Walk* de la **Parte 3** (porque se ejecuta en circuitos dedicados en lugar de instrucciones de software), sigue siendo necesario consultar la tabla de páginas en memoria para cada traducción.

La diferencia clave entre el *Page Table Walk* por software (**Parte 3**) y el *Page Table Walk* por hardware (**Parte 4**, **Escenario A**) radica en la implementación. En el primer caso, el sistema operativo ejecuta código (instrucciones de CPU) para recorrer la tabla de páginas, lo que implica múltiples accesos a memoria y ciclos de CPU. En el segundo caso, la MMU posee circuitos combinacionales y secuenciales dedicados que realizan la misma operación en muchos menos ciclos.

Los mensajes del simulador lo reflejan, en la **Parte 3**:

```
[OS] Software Page Table Walk in progress ........ [DONE]
```

Con más puntitos (indicando más ciclos), mientras que en este escenario (**Parte 4**) aparece:

```
[MMU] Hardware Page Table Walk in progress ... [DONE] con menos puntos.
```

La limitación fundamental de este escenario es que cada acceso a memoria requiere un recorrido de la tabla de páginas, sin importar si la misma página fue accedida recientemente. La mejora natural consiste en agregar una caché de traducciones (**TLB**) que almacene los mapeos `VPN` ← `PFN` más recientes.
De esta forma, los accesos repetidos a una misma página se resolverían en un solo ciclo (**TLB Hit**), y solo los accesos a páginas nuevas o desalojadas requerirían un *Page Table Walk*. Este es precisamente el tema del **Escenario B**.

Escenario B: TLB Multiranura

Se programó el **TLB** en el archivo `tlb.c` con las funciones `tlb_lookup` y **tlb_insert**. El *TLB* implementado tiene capacidad para 5 *slots* y utiliza el algoritmo **LRU** (*Least Recently Used*) para decidir qué entrada desalojar cuando el **TLB** está lleno, con **FIFO** como criterio de desempate. Cada entrada almacena el número de página virtual, el marco físico correspondiente, un contador de último acceso y un *timestamp* de inserción. Se validó el código y se habilitó el bloque completo (**MMU** + **TLB**) en el simulador.

```c
#include <stdio.h>
#include "tlb.h"

tlb_entry_t tlb[TLB_SIZE]; //Arreglo de entradas del TLB (cache de
    traducciones)
int global_timer = 0; //Reloj simulado para LRU/FIFO

/* Inicializa todas las entradas del TLB como invalidas */
void
init_tlb()
{
  for (int i = 0; i < TLB_SIZE; i++) {
    tlb[i].valid = 0;
    tlb[i].page = -1;
    tlb[i].frame = -1;
    tlb[i].last_access = 0;
    tlb[i].insertion_time = 0;
  }
}

/*
 * Busca una pagina en el TLB.
 * Recorre todas las entradas del TLB buscando una entrada valida
 * que coincida con el numero de pagina solicitado.
 */
int
tlb_lookup(int page)
{
  global_timer++;
  for (int i = 0; i < TLB_SIZE; i++) {
    if (tlb[i].valid && tlb[i].page == page) {
```

```c
      tlb[i].last_access = global_timer;  //Actualizamos para politica LRU
      printf("TLB HIT: Page %d found in Entry %d (Frame %d)\n", page, i, tlb[i].frame);
      return tlb[i].frame;
    }
  }

  // Si no se encuentra (MISS), notificar y retornar -1
  printf("TLB MISS: Page %d not in TLB\n", page);
  return -1;
}


/*
 * Inserta una traduccion pagina->frame en el TLB
 * Estrategia de insercion:
 *   1. Buscamos una entrada vacia (valid == 0) y se inserta ahi.
 *   2. Si el TLB esta lleno, aplicamos desalojo LRU:
 *      - Seleccionamos la entrada con menor last_access (menos recientemente usada).
 *      - En caso de empate en last_access, usamos FIFO como desempate,
 *        desalojamos la que fue insertada primero (menor insertion_time).
 */
void
tlb_insert(int page,
       int frame)
{
  global_timer++;

  //Caso 1: Buscamos entrada vacia para insertar sin desalojar
  for (int i = 0; i < TLB_SIZE; i++) {
    if (!tlb[i].valid) {
      tlb[i].valid = 1;
      tlb[i].page = page;
      tlb[i].frame = frame;
      tlb[i].last_access = global_timer;
      tlb[i].insertion_time = global_timer;
      printf("TLB INSERT: Page %d -> Frame %d (Entry %d)\n", page, frame, i);
      return;
    }
  }

  //Caso 2: TLB lleno, seleccionamos victima con LRU + FIFO como desempate
  int victim = 0;
  for (int i = 1; i < TLB_SIZE; i++) {
    if (tlb[i].last_access < tlb[victim].last_access)
      victim = i;
```

```c
    else if (tlb[i].last_access == tlb[victim].last_access)
      //Empate en LRU: desalojar el que se inserto primero (FIFO)
      if (tlb[i].insertion_time < tlb[victim].insertion_time)
    victim = i;
  }
  printf("TLB EVICT: Entry %d (Page %d) replaced\n", victim, tlb[
    victim].page);

  //Sobreescribimos la entrada victima con la nueva traduccion
  tlb[victim].valid = 1;
  tlb[victim].page = page;
  tlb[victim].frame = frame;
  tlb[victim].last_access = global_timer;
  tlb[victim].insertion_time = global_timer;
  printf("TLB INSERT: Page %d -> Frame %d (Entry %d)\n", page, frame,
    victim);
}
```

Código 4: Implementación de tlb `tlb.c`

Se ejecutó un proceso `taskA` de **100 KB** instrucción por instrucción. La ejecución completa requirió aproximadamente 26 pasos, un número similar al **Escenario A**. Sin embargo, la diferencia fundamental no radica en sí en el número de pasos, sino en el costo de cada paso.

El panel de estadísticas del **TLB** muestra los siguientes resultados al finalizar la ejecución:

12 hits, 14 misses, un ratio de acierto del 46.2% y un total de 3900 ciclos.

El **TLB** tiene 5 ranuras ocupadas al final de la ejecución, con las páginas virtuales 21, 22, 4, 13 y 24 mapeadas a los marcos físicos 28, 29, 11, 20 y 31 respectivamente.

De los 26 accesos totales a memoria, 12 se resolvieron directamente desde el **TLB** en un solo ciclo cada uno (**TLB Hit**), mientras que los 14 restantes requirieron un *Hardware Page Table Walk* completo (**TLB Miss**), un **TLB Hit** tiene un costo de apenas 1 ciclo, como lo confirma el mensaje:

```
[MMU HIT] Physical Address found. Cycle cost: 1.
```

En contraste, un **TLB Miss** implica un recorrido completo de la tabla de páginas, con un costo significativamente mayor (decenas de ciclos). El ratio de 46.2% puede parecer modesto, pero representa un ahorro sustancial, en el **Escenario A** (sin **TLB**), los 26 accesos habrían requerido 26 *Page Table Walks* completos. Con el **TLB**, 12 de esos accesos se resolvieron instantáneamente, eliminando casi la mitad de los recorridos de tabla.

Al analizar la traza de ejecución, se aprecia un patrón característico. Las primeras instrucciones generan misses porque el **TLB** comienza vacío y debe poblarse gradualmente. Una vez que una página se carga en el **TLB**, los accesos subsecuentes a la misma página producen hits, como se observa cuando la instrucción 4 accede nuevamente al `VPN 5` (que ya había sido cargado en la instrucción 1) y obtiene un hit.

Sin embargo, dado que el **TLB** tiene solo 5 ranuras y el proceso accede a más de 5 páginas distintas, inevitablemente ocurren desalojos, una página desalojada que vuelve a ser accedida genera un nuevo miss. Los bucles (*Branch Loop*) del programa producen accesos repetidos a un subconjunto de páginas, lo que favorece los hits dentro de cada iteración del bucle.

Principio de Localidad de Referencia

La **Versión A** (recorrido por filas) ofrece un rendimiento significativamente superior en términos de latencia, la razón se fundamenta en cómo interactúan el patrón de acceso con la organización de la memoria y el **TLB**.

En *Row-Major Order*, los elementos de una misma fila se almacenan en posiciones de memoria contiguas, loq ue significa que al recorrer fila por fila (**Versión A**), los accesos consecutivos tocan posiciones de memoria adyacentes, que con alta probabilidad pertenecen a la misma página. El primer acceso a cada página genera un **TLB Miss** y carga la traducción en el **TLB**, pero todos los accesos subsecuentes a elementos de la misma fila (y por tanto de la misma página) se resuelven como **TLB Hits** de 1 ciclo. De este modo se aprovecha la localidad espacial, la tendencia de un programa a acceder a direcciones de memoria cercanas a las que ha accedido recientemente.

Además, la **Versión A** se beneficia de la localidad temporal dentro de cada fila, la traducción recién cargada en el **TLB** se reutiliza múltiples veces antes de pasar a la siguiente fila, maximizando la vida útil de cada entrada del **TLB**.

En contraste, la **Versión B** (recorrido por columnas) accede a un elemento de la primera fila, luego a un elemento de la segunda fila, luego de la tercera, y así sucesivamente.

En *Row-Major Order*, estos elementos están separados por distancias iguales al ancho completo de la fila en memoriam, si la matriz es grande, cada acceso consecutivo cae en una página diferente, generando un **TLB Miss** en cada operación. Para cuando el recorrido regresa a la primera fila (al pasar a la segunda columna), las traducciones originales ya fueron desalojadas del **TLB** por las de otras filas, generando nuevos misses; el resultado es un ratio de acierto cercano a cero y una latencia drásticamente mayor.

Si consideramos una matriz de 1000×1000 elementos de 4 bytes con páginas de **4 KB** (1024 elementos por página). La **Versión A** genera aproximadamente `1000` **TLB Misses** (uno por cada página nueva al inicio de cada grupo de aprox 1024 elementos) y 999×1000 **TLB Hits**; mientras que la **Versión B** genera hasta 1×10^6 **TLB Misses** si el **TLB** no tiene capacidad suficiente para retener las traducciones entre iteraciones de columna, la diferencia en rendimiento puede ser de un orden de magnitud o más.

Del principio de localidad de referencia se derivan varias directrices prácticas para optimizar el acceso a memoria en programas reales:

- Recorrer las estructuras de datos en el mismo orden en que están almacenadas en memoria. En C (Row-Major), iterar primero sobre columnas (variando el índice más interno) dentro de cada fila (variando el índice más externo).
- Organizar los datos de manera que los elementos accedidos juntos se almacenen en posiciones contiguas. Si un algoritmo procesa un subconjunto de campos de un arreglo de estructuras, puede ser más eficiente usar arreglos separados por campo (*Structure of Arrays*) en lugar de un arreglo de estructuras (*Array of Structures*).
- Cuando un algoritmo opera sobre bloques o submatrices, dimensionar los bloques para que quepan en el **TLB** (y en las cachés `L1`/`L2`). Esta técnica, conocida como *blocking* o *tiling*, maximiza la reutilización de las traducciones cargadas antes de que sean desalojadas.
- Evitar patrones de acceso con saltos grandes entre direcciones de memoria (*strides* largos), ya que cada salto puede cruzar un límite de página y provocar un **TLB Miss** innecesario.
- En algoritmos de recorrido de grafos o estructuras enlazadas, agrupar los nodos relacionados en regiones contiguas de memoria (mejora de localidad por diseño de datos) siempre que sea posible.

Parte 5.

```
        TOKEN: ECBBFA9E2D279932E26363A308BC198B

PHASE: Basic Paging (pag-basic)
    [OK]   Check #1 passed.
    [OK]   Check #2 passed.
    RESULT: PASS
        TOKEN: 65F4FB5B4B5B6718DACF6D3575819452

PHASE: MMU Basic Translation (mmu-hw)
    [OK]   Check #1 passed.
    [OK]   Check #2 passed.
    RESULT: PASS
        TOKEN: E0EDC84E298C9A0B37976336D6C4C176

PHASE: TLB with LRU/FIFO (tlb-fast)
    [OK]   Check #1 passed.
    [OK]   Check #2 passed.
    [OK]   Check #3 passed.
    RESULT: PASS
        TOKEN: 7CFA17432642E45F48DBCCCEB83F88A2

PHASE: Demand Paging (pg-od)
    [OK]   Check #1 passed.
    [OK]   Check #2 passed.
    [OK]   Check #3 passed.
    [OK]   Check #4 passed.
    RESULT: PASS
        TOKEN: F372AA7C4AAE54B77A970FB6482170C4

==========================================
 [src/] >
```

5.1 Implementación:

Se programó la función de paginación bajo demanda en el archivo `paging_on_demand.c`, implementando las funciones `find_free_frame` (búsqueda de marcos libres), `evict_frame` (desalojo **LRU** global excluyendo marcos de tablas de páginas), `load_process` (carga de proceso con tabla de páginas fijada en RAM) y `access_page` (gestión de hits y page faults). Se validó el código y se obtuvo el *Enabler Code*, que se ingresó en el simulador para habilitar el módulo.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "paging_on_demand.h"

#define MAX_FRAMES 128
#define MAX_PROCESSES 10
#define MAX_PAGES 64

typedef struct {
  int pid;
  int v_page;
  int is_page_table;
  int last_use;
} frame_info_t;

frame_info_t ram_frames[MAX_FRAMES];
int total_ram_frames = 0;
int demand_paging_timer = 0;

typedef struct {
  int pid;
  int page_table[MAX_PAGES]; // Maps v_page to frame index (-1 if not in RAM)
  int total_pages;
} process_demand_t;

process_demand_t demand_processes[MAX_PROCESSES];
int demand_process_count = 0;

/*
 * Inicializamos el subsistema de paginacion bajo demanda.
 * Limpiamos todos los marcos y reinicia el contador de procesos.
 */
void
init_demand_system(int frames)
{
  total_ram_frames = (frames > MAX_FRAMES) ? MAX_FRAMES : frames;
  for (int i = 0; i < MAX_FRAMES; i++) {
    ram_frames[i].pid = -1;
    ram_frames[i].is_page_table = 0;
    ram_frames[i].last_use = 0;
  }
  demand_process_count = 0;
  printf("SYSTEM: Demand Paging initialized with %d frames\n", total_ram_frames);
}

/*
 * Buscamos el primer marco fisico libre en RAM.
 * Recorremos secuencialmente los marcos hasta encontrar uno con pid
```

```
    == -1.
 */
int
find_free_frame()
{
  for (int i = 0; i < total_ram_frames; i++) {
    if(ram_frames[i].pid == -1)
      return i;
  }
  return -1;
}

/*
 * Desalojamos un marco usando la politica LRU Global.
 * Seleccionamos el marco con el menor valor de last_use (el menos
 * recientemente usado) como victima para desalojo.
 * Los marcos marcados como is_page_table == 1 estan protegidos (
    pinned)
 * y nunca se desalojan, ya que perder una tabla de paginas
    corromperia todo el proceso.
 * Tras seleccionar la victima:
 *   - Invalida la entrada correspondiente en la tabla de paginas del
    proceso
 *   - Limpia el marco para reutilizacion
 */
int
evict_frame()
{
  int victim = -1;
  int oldest_use = __INT_MAX__;

  //LRU Global: buscamos el frame con menor last_use, excluyendo
    tablas de paginas
  for (int i = 0; i < total_ram_frames; i++) {
    if (ram_frames[i].pid != -1 && ram_frames[i].is_page_table == 0) {
      if (ram_frames[i].last_use < oldest_use) {
    oldest_use = ram_frames[i].last_use;
    victim = i;
      }
    }
  }
  if (victim != -1) {
    printf("EVICT: Frame %d (PID %d, Page %d) removed from RAM\n",
       victim, ram_frames[victim].pid, ram_frames[victim].v_page);
    //Invalidamos la entrada en la tabla de paginas del proceso
    afectado
    for (int i = 0; i < demand_process_count; i++) {
      if (demand_processes[i].pid == ram_frames[victim].pid) {
    demand_processes[i].page_table[ram_frames[victim].v_page] = -1;
    break;
      }
    }
```

```c
    //Limpiar el marco para que pueda ser reutilizado
    ram_frames[victim].pid = -1;
    ram_frames[victim].v_page = -1;
    ram_frames[victim].is_page_table = 0;
    ram_frames[victim].last_use = 0;
  }
  return victim;
}


/*
 * Registra un nuevo proceso y reserva su tabla de paginas.
 * Reserva un marco exclusivo para la tabla de paginas del proceso y lo
 * marca como pinned (is_page_table = 1) para que nunca sea desalojado.
 * Las paginas de datos se cargan bajo demanda mediante access_page.
 */
void
load_process(int pid,
         int pages)
{
  if (demand_process_count >= MAX_PROCESSES)
    return;
  //Reservamos un frame para la tabla de paginas del proceso
  int pt_frame = find_free_frame();
  if (pt_frame == -1)
    pt_frame = evict_frame();
  if (pt_frame != -1) {
    //Marcamos el frame como tabla de paginas (pinned, no se desaloja)
    ram_frames[pt_frame].pid = pid;
    ram_frames[pt_frame].v_page = -1;
    ram_frames[pt_frame].is_page_table = 1;
    ram_frames[pt_frame].last_use = ++demand_paging_timer;
    //Inicializamos el proceso
    demand_processes[demand_process_count].pid = pid;
    demand_processes[demand_process_count].total_pages = pages;
    for (int i = 0; i < MAX_PAGES; i++)
      demand_processes[demand_process_count].page_table[i] = -1;
    printf("LOAD: PID %d loaded. Page Table pinned at Frame %d\n", pid, pt_frame);
    demand_process_count++;
  }
}

/*
 * Simula el acceso a una pagina virtual de un proceso.
 * Flujo de ejecucion:
 *   1. Buscar el proceso en el arreglo de procesos registrados.
 *   2. Consultar la tabla de paginas para verificar si la pagina esta en RAM.
 *   3. Si esta (HIT): actualizar el timestamp last_use para LRU.
```

```c
 *   4. Si no esta (PAGE FAULT):
 *      a. Buscar un marco libre o desalojar uno (LRU).
 *      b. Cargar la pagina desde disco al marco (PAGE IN).
 *      c. Actualizar la tabla de paginas con el nuevo mapeo.
 */
void
access_page(int pid,
        int v_page)
{
  demand_paging_timer++;
  //Buscamos el proceso
  process_demand_t *proc = NULL;
  for (int i = 0; i < demand_process_count; i++) {
    if (demand_processes[i].pid == pid) {
      proc = &demand_processes[i];
      break;
    }
  }
  if (proc == NULL) {
    printf("ERROR: PID %d not found\n", pid);
    return;
  }
  //Verificamos si la pagina esta en RAM
  int frame = proc->page_table[v_page];
  if (frame != -1 && ram_frames[frame].pid == pid && ram_frames[frame].v_page == v_page) {
    //HIT: pagina en RAM, actualizamos last_use
    ram_frames[frame].last_use = demand_paging_timer;
    printf("ACCESS: PID %d, Page %d -> Hit at Frame %d\n", pid, v_page, frame);
  } else {
    //PAGE FAULT: pagina no esta en RAM
    printf("PAGE FAULT: PID %d, Page %d not in RAM\n", pid, v_page);
    //Buscamos frame libre o desalojar uno
    int new_frame = find_free_frame();
    if (new_frame == -1)
      new_frame = evict_frame();
    if (new_frame != -1) {
      //Cargamos la pagina en el frame
      ram_frames[new_frame].pid = pid;
      ram_frames[new_frame].v_page = v_page;
      ram_frames[new_frame].is_page_table = 0;
      ram_frames[new_frame].last_use = demand_paging_timer;
      //Actualizamos tabla de paginas del proceso
      proc->page_table[v_page] = new_frame;
      printf("PAGE IN: PID %d, Page %d loaded into Frame %d\n", pid, v_page, new_frame);
    }
  }
}

/*
```

```c
 * Punto de entrada del simulador */
void
simulate_demand_paging()
{
  char line[256];
  while (fgets(line, sizeof(line), stdin)) {
    line[strcspn(line, "\r\n")] = 0;
    if (line[0] == '\0' || line[0] == '#') continue;
    int val, pid, pages, v_page;
    if (sscanf(line, "RAM_CONFIG FRAMES %d", &val) == 1)
      init_demand_system(val);
    else if (sscanf(line, "LOAD PID %d PAGES %d", &pid, &pages) == 2)
      load_process(pid, pages);
    else if (sscanf(line, "ACCESS PID %d V_PAGE %d", &pid, &v_page) ==
    2)
      access_page(pid, v_page);
  }
}
```

Código 5: Implementación paging on demand `paging_on_demand.c`

5.2 Carga Parcial:

Se utilizaron los siguientes comandos para crear procesos con carga parcial de páginas, con un tamaño de página de **4 KB** (256 marcos totales):

`exec taskB 150 --initial-pages 15`

Este comando crea un proceso de **150 KB** que requiere 38 páginas de **4 KB**, pero solo carga las primeras 15 en marcos físicos de RAM. Las 23 páginas restantes quedan almacenadas en disco (*Disk Swap*).

`exec taskC 128 --initial-pages 3`

Este comando crea un proceso de **128 KB** que requiere 32 páginas, pero solo carga 3 en RAM, las 29 restantes permanecen en disco.

El efecto más evidente en el visualizador de memoria es la diferencia entre el tamaño total del proceso y el espacio que realmente ocupa en RAM. Para `taskB`, a pesar de ser un proceso de **150 KB** (38 páginas), únicamente 15 marcos físicos aparecen asignados en el visualizador; el espacio restante no se consume, para `taskC`, de **128 KB** (32 páginas), solo 3 marcos aparecen ocupados en la RAM.

La tabla de páginas (`PAGE TABLE DUMP`) refleja esta situación con claridad, las páginas cargadas en RAM muestran un **PFN** asignado (por ejemplo, `F15, F16, F17...`) con el bit de presencia (`P-Bit`) en 1. Las páginas no cargadas muestran "**DISK**" como ubicación y **P-Bit** en 0, indicando que existen en el almacenamiento secundario pero

no ocupan espacio en memoria física.

Con `-initial-pages` se nos permite controlar la huella de memoria inicial de un proceso:

- Un valor bajo (como 3) significa que el proceso arranca consumiendo muy poca RAM, dejando marcos disponibles para otros procesos. A cambio, las páginas que el proceso necesite durante su ejecución y que no estén en RAM provocarán fallos de página, que deberán resolverse cargando la página desde disco.
- Un valor alto (como 128) precarga más páginas, reduciendo los futuros page faults pero consumiendo más RAM desde el inicio.

Esta flexibilidad es lo que permite a los sistemas operativos modernos ejecutar múltiples procesos cuya suma de tamaños excede la RAM disponible, cada proceso solo ocupa los marcos que realmente está utilizando en un momento dado, no su espacio de direcciones completo.

5.3 Page Faults:

Se seleccionó el proceso `taskC` (`PID=107`, creado con `-initial-pages 3`) y se ejecutaron instrucciones paso a paso mediante `step`, las páginas 0, 1 y 2 son las únicas presentes en RAM; todas las demás (3 a 31) residen en disco.

Al ejecutar el primer `step`, la instrucción accede al `VPN 15`, que se encuentra en RAM (fue una de las primeras páginas cargadas), la ejecución procede normalmente. Sin embargo, al ejecutar el segundo `step`, la instrucción intenta acceder al `VPN 9`, que no está en RAM (su estado es `DISK`). En ese momento, el simulador detiene la ejecución y

muestra el mensaje:

```
PAGE FAULT DETECTED VPN 9 is in Disk Swap.
```

```
Use "page-in 9" (or resolve buttons) to fetch it.
```

Este mensaje indica que se disparó un fallo de página, la MMU detectó que la página solicitada tiene el bit de presencia en 0, lo que significa que no está en memoria física, la ejecución del proceso se suspende hasta que la página sea cargada desde disco.

El comando `page-in <vpn>` permite resolver manualmente un fallo de página. Al ejecutar:

```
page-in 9
```

El kernel busca un marco físico libre, copia el contenido de la página desde disco a ese marco, actualiza la tabla de páginas del proceso (asignando el **PFN** y poniendo el `P-Bit` en 1), y permite que la ejecución continúe; el simulador confirma la operación con el mensaje:

```
[KERNEL] Page Fault for PID 107, VPN 9. Loading from disk... Page (VPN 9) loaded into
```

En un sistema operativo real, este proceso ocurre automáticamente, el hardware genera una interrupción (*trap*) al detectar el *page fault*, el kernel ejecuta la rutina de manejo de fallos de página que carga la página desde disco, y la instrucción que provocó el fallo se reintenta de forma transparente para el proceso.

Un detalle importante es que tras resolver el *page fault* del `VPN 9` y continuar con `step`, la siguiente instrucción puede acceder nuevamente al `VPN 9` sin problema (la página ya está en RAM). Sin embargo, si la siguiente instrucción accede a otro **VPN** que también está en disco (por ejemplo, `VPN 8`), se dispara un nuevo `page fault` que debe resolverse de la misma manera.

5.4 Evicción y Desalojo (Dirty Bit)::

Se llevó la RAM a su capacidad máxima mediante la creación de dos procesos de **512 KB** cada uno con 128 páginas iniciales, posteriormente se utilizó el comando `evict <pfn>` para liberar marcos físicos específicos y observar el comportamiento del sistema.

Al ejecutar `evict` sobre distintos marcos, se observaron tres escenarios:

- Página limpia (`Dirty = 0`): Al ejecutar `evict 32`, el kernel muestra el mensaje:

  `"Frame 32 evicted. Page (VPN 0) of PID 113 dropped (not dirty)".`

  La página se descarta directamente de RAM sin necesidad de respaldarla a disco, ya que su contenido no fue modificado desde que se cargó, la copia en disco sigue siendo válida. Este tipo de evicción es rápido porque no requiere operaciones de escritura en disco.

- Marco de tabla de páginas (`pinned`): Al intentar `evict 0`, el kernel responde con el error

  `"Cannot evict Page Table overhead frame".`

  Los marcos que almacenan las tablas de páginas de los procesos están protegidos (fijados) y no pueden ser desalojados bajo ninguna circunstancia. Si se desalojara la tabla de páginas de un proceso, el sistema perdería toda referencia sobre dónde se encuentran sus páginas, lo que provocaría una corrupción catastófica.

- Página sucia (`Dirty = 1`) (análisis teórico): Durante las pruebas experimentales, todas las instrucciones generadas por los procesos fueron de tipo READ, por lo que no se logró generar alguna página marcada como sucia. No obstante, en un escenario donde un proceso ejecute instrucciones de escritura (`WRITE`), la página modificada se marcaría con `Dirty = 1` en la tabla de páginas. Al intentar desalojar una página sucia, el kernel debe primero copiar su contenido actualizado al disco (`write-back`) antes de liberar el marco, ya que la copia en disco está desactualizada. Este paso adicional de escritura a disco hace que la evicción de páginas sucias sea significativamente más costosa que la de páginas limpias.

El dirty bit es un indicador de un solo bit en cada entrada de la tabla de páginas que el hardware actualiza automáticamente cada vez que se realiza una escritura sobre esa página, su propósito es optimizar el proceso de evicción, si una página no fue modificada (`dirty = 0`), el kernel sabe que la copia en disco es idéntica a la copia en RAM, por lo que puede descartar la copia en RAM sin hacer nada más. Si fue modificada (`dirty = 1`), el kernel debe sincronizar la copia en disco antes de liberar el marco, esta distinción permite al sistema evitar escrituras innecesarias a disco, que son operaciones extremadamente lentas comparadas con los accesos a RAM.

5.5 Conclusión Técnica:

Con la paginación bajo demanda, la capacidad de carga del sistema parece expandirse más allá del límite físico de la RAM. En las pruebas realizadas, se crearon dos procesos de **512 KB** cada uno (**1024 KB** en total, el 100% de la RAM) con solo 128 páginas iniciales cada uno, dejando las páginas restantes en disco. Inclusive, fue posible crear un tercer proceso con `-initial-pages` bajo sin agotar la RAM, ya que solo las páginas efectivamente cargadas consumen marcos físicos.

En teoría, el sistema podría crear decenas de procesos cuyo tamaño total combinado supere varias veces la RAM disponible, siempre que no todos necesiten todas sus páginas simultáneamente en memoria.

Ventajas:

- **Multiprogramación ampliada:** se pueden ejecutar más procesos simultáneamente, ya que cada uno consume solo la memoria que necesita en cada momento, no su espacio de direcciones completo.
- **Inicio rápido:** los procesos pueden comenzar a ejecutarse inmediatamente sin esperar a que todas sus páginas se carguen en RAM, solo se cargan las páginas que el proceso accede realmente.
- **Ejecución de programas grandes:** es posible ejecutar programas cuyo tamaño total excede la RAM física disponible, algo imposible en los esquemas anteriores.
- **Eficiencia de recursos:** las páginas de código que nunca se ejecutan (por ejemplo, rutinas de manejo de errores que no se activan) nunca se cargan en RAM, ahorrando marcos para páginas que sí se necesitan.

Desventajas

- **Latencia por page faults:** cada fallo de página implica una transferencia desde disco, cuyo costo es de varios órdenes de magnitud superior a un acceso a RAM. Si los page faults son frecuentes, el rendimiento se degrada severamente.
- **Thrashing:** si la cantidad de marcos físicos es insuficiente para el conjunto de trabajo activo de los procesos (*working set*), el sistema entra en un estado donde gasta más tiempo desalojando y cargando páginas que ejecutando instrucciones útiles. Este fenómeno, conocido como *thrashing*, puede reducir el rendimiento del sistema a una fracción mínima de su capacidad.
- **Complejidad del sistema:** el manejo de *page faults*, el algoritmo de reemplazo (**LRU**), el control del *dirty bit* y la gestión de marcos fijados añaden una complejidad significativa al kernel del sistema operativo.
- **Dependencia del disco:** el rendimiento está fuertemente ligado a la velocidad del almacenamiento secundario, con discos **HDD** tradicionales, los *page faults* son extremadamente costosos; con **SSDs** o **NVMe**, el impacto se reduce pero sigue siendo sustancial comparado con accesos a RAM.

A pesar de la expansión aparente, existen límites concretos observados durante la experimentación, la RAM física sigue siendo el recurso crítico, cuando todos los marcos están ocupados, cada *page fault* requiere desalojar una página existente, lo que puede

generar *page faults* en cadena si la página desalojada es necesitada poco después.

Los marcos de tablas de páginas no pueden ser desalojados (son fijos), lo que significa que cada proceso nuevo consume al menos un marco permanente solo para su tabla, reduciendo progresivamente los marcos disponibles para páginas de datos. Finalmente, el presupuesto real de RAM no cambia, la paginación bajo demanda es una estrategia de administración inteligente del recurso, no una forma de crear memoria adicional.

En síntesis, la paginación bajo demanda intercambia tiempo (latencia de acceso a disco) por espacio (capacidad aparente de memoria), y su eficacia depende de que los programas exhiban buena localidad de referencia, de modo que el conjunto de páginas activamente utilizadas quepa en los marcos disponibles.

Conclusión

A lo largo de esta práctica recorrimos la evolución completa de la gestión de memoria en los sistemas operativos , desde el esquema más primitivo (el acceso directo a RAM) hasta el modelo de memoria virtual con paginación bajo demanda que utilizan los sistemas modernos. Cada etapa surgió como respuesta a limitaciones concretas de la etapa anterior, y los experimentos realizados en el simulador `RAM-Sim` permitieron observar esas limitaciones y las soluciones implementadas.

El punto de partida en la **Parte 1** fue el acceso directo a la memoria física, un esquema sin abstracciones ni protecciones, los experimentos revelaron dos deficiencias graves, la fragmentación externa, que impedía aprovechar la memoria libre cuando estaba dispersa en bloques no contiguos, y la ausencia total de aislamiento entre procesos, que permitía a cualquier proceso leer o modificar la memoria de otros, lo que mostró lo

crítico que resulta este problema en un sistema sin protección.

La introducción de los registros Base y Límite en la **Parte 2** resolvió el problema de seguridad, al asignar a cada proceso un rango válido de direcciones, el hardware intercepta cualquier acceso fuera de ese rango y genera una falla de segmentación. Los experimentos confirmaron este comportamiento, al intentar que `taskA` accediera a la memoria de `taskB`, el simulador abortó la operación con el error *Segmentation Violation*, sin embargo, este mecanismo no resolvió la fragmentación externa ni permitió la compartición controlada de memoria.

La paginación básica de la **Parte 3** eliminó la fragmentación externa al dividir la memoria en marcos de tamaño fijo, un proceso ya no necesita un bloque contiguo, sus páginas pueden distribuirse en cualquier marco disponible. Los experimentos mostraron que un proceso pudo cargarse con éxito en escenarios donde antes el sistema reportaba *Out of Memory*, y que la compartición segura de marcos entre procesos era ahora posible mediante los comandos `add-owner`, `remove-owner` y `chmod`. Como contrapartida, la paginación introdujo el *Page Table Walk* en cada acceso a memoria, lo que multiplicó la latencia.

La **Parte 4** mostró la importancia decisiva de la **MMU** y el **TLB** en el rendimiento de la paginación, la **MMU** por hardware redujo la ejecución de un proceso de alrededor de 50 pasos (paginación por software) a aproximadamente 26 pasos, y el **TLB** elevó aún más la eficiencia al cachear las traducciones recientes, con un *ratio* de aciertos del 46.2% en las pruebas, lo que permitió que casi la mitad de los accesos se resolvieran en un solo ciclo. El análisis del principio de localidad de referencia reveló que el rendimiento del **TLB** depende críticamente del patrón de acceso, un recorrido por filas en una matriz almacenada en *Row-Major Order* puede alcanzar tasas de acierto superiores al 95%, mientras que un recorrido por columnas destruye la localidad y provoca misses en casi cada acceso.

Finalmente, la paginación bajo demanda de la **Parte 5** completó el modelo al permitir que los procesos inicien con solo un subconjunto de sus páginas en RAM, los experimentos con `-initial-pages` mostraron cómo un proceso de **150 KB** podía ejecutarse con apenas 15 páginas iniciales, cargando las demás bajo demanda mediante el mecanismo de *page faults*. Por otro lado, la mecánica de evicción permitió concluir que la distinción fundamental entre páginas limpias y sucias para el rendimiento del sistema, una página limpia (`Dirty = 0`) puede descartarse de RAM de forma inmediata porque su copia en disco sigue siendo válida, mientras que una página sucia (`Dirty = 1`) obliga al kernel a realizar un *write-back* a disco antes de liberar el marco, una operación considerablemente más costosa. Asimismo, se confirmó la importancia de proteger las estructuras críticas del sistema mediante marcos fijados, cuyo desalojo provocaría la pérdida total de las referencias de un proceso a sus páginas en memoria.

Esta evolución etapa por etapa surgió para resolver un problema específico, la falta de protección es la razón del *Base-Límite*, la fragmentación externa motivó la paginación, la latencia del *Page Table Walk* motivó la **MMU** por hardware y el **TLB**, y la limitación de la RAM física motivó la paginación bajo demanda. Cada solución introdujo complejidad adicional, pero también permitió construir sistemas más seguros, eficientes y capaces.
El resultado final es un sistema donde el software (sistema operativo) y el hardware (**MMU**, **TLB** , *dirty bit*, bit de presencia) colaboran para ofrecer a cada proceso la ilusión de un espacio de memoria privado, protegido, continuo y potencialmente mayor que la memoria física real.